

ARCHITECTURE REVIEW & ENGINEERING
CONSULTATION

PDOS Price Delivery Operating System

A senior-architect review of PDOS as an emerging Market Operating System — evaluating registry philosophy, replay correctness, rendering architecture, and multi-timeframe synchronization. Recommendations preserve swing and liquidity mathematics, the SQLite schema, and the TradingView Lightweight Charts surface.

PREPARED BY

Senior Software Architect

ENGAGEMENT

Architecture Review & Engineering Consultation

DATE

30 June 2026

SCOPE

Backend · Frontend · Replay · Rendering

Preserve Information · Interpret Later

Never lose market facts

Table of Contents

Chapter 1 — Executive Summary & Engagement Framing	3
Three findings that matter most	3
Chapter 2 — Current Architecture Review	4
2.1 The pipeline, end to end	4
2.2 Mapping to established patterns	5
2.3 Registry identity and the "one swing = one liquidity object" rule	5
Chapter 3 — Strong Points	6
3.1 Registry-first design preserves market truth	6
3.2 Immutable swings enable time-travel debugging	6
3.3 Clean separation of fact from interpretation	6
3.4 SQLite as append-mostly event log	7
3.5 TVLC as rendering surface, not state store	7
3.6 Layered rendering pipeline	7
Chapter 4 — Weak Points	7
4.1 Render Scheduler is doing two jobs	7
4.2 No explicit event bus	8
4.3 Render Index invalidation granularity is unclear	8
4.4 DrawingCanvas is a monolithic boundary	8
4.5 Multi-timeframe synchronization has no transactional contract	9
4.6 No feature flags visible in the architecture	9
Chapter 5 — Performance Bottlenecks	9
5.1 Backend: SQLite query plans	10
5.2 Backend: replay throughput vs symbol count	10
5.3 Frontend: TVLC render budget	10
5.4 Frontend: Render Index rebuild on seek	11
Chapter 6 — Rendering Bottlenecks	11
6.1 DrawingCanvas vs Series Primitives — the trade-off	11

6.2 PrimitiveManager — the abstraction	12
6.3 Layer Manager overhead at high layer count	12
6.4 Projection Service cost on multi-timeframe	13
6.5 TVLC pan/zoom repaint cost	13
Chapter 7 — Maintainability Issues	13
7.1 Module boundaries are leaking	13
7.2 No characterization tests pinning current behavior	14
7.3 Rendering layer is the most volatile and least tested	14
7.4 Naming consistency across render components	14
7.5 ARCHITECTURE.md does not exist	14
Chapter 8 — Scalability Issues	15
8.1 Symbol count — the 1000-symbol question	15
8.2 Timeframe count — the $O(TF^2)$ coordination problem	15
8.3 History depth — SQLite partitioning	15
8.4 Concurrent replays — per-session isolation	15
8.5 Future engine count — the RegistryReader contract	16
Chapter 9 — Architectural Improvements	16
9.1 EventBus — typed channels for cross-domain communication	16
9.2 PrimitiveManager — per-layer canvas/primitive opt-in	17
9.3 TransactionManager — begin/commit/rollback over the replay cursor	17
9.4 RegistryReader — the read-only port for future engines	17
9.5 FeatureFlags — off / shadow / on	18
9.6 Characterization test protocol	19
Chapter 10 — Future Roadmap	19
10.1 Sequencing rationale	20
Chapter 11 — Risk Assessment	20
11.1 Three iron rules	22

Chapter 1 — Executive Summary & Engagement Framing

This document is a senior-architect review of the PDOS (Price Delivery Operating System) project. It is written from the brief supplied by the project owner, supplemented by widely known patterns of registry-based, event-sourced trading systems. It does not assume access to the source tree; where file-level specificity would change a recommendation, that limitation is called out explicitly.

Engagement framing. The Google Drive folder referenced in the brief is outside my reachable workspace, so I cannot read the actual PDOS codebase. The review that follows is therefore architectural-pattern-level rather than file-specific. It is grounded in the project description supplied: the Swing Engine → Swing Registry → Liquidity Registry → Pipeline → SQLite → API → Frontend → TradingView Lightweight Charts (TVLC) → Rendering Layer data flow, and the named rendering components (Projection Service, Render Scheduler, Render Index, Layer Manager, DrawingCanvas). Where a recommendation depends on a code-level detail I cannot verify, I mark it as a hypothesis to validate rather than a finding.

My stance, stated up front. PDOS is not a trading application, an indicator, or a charting tool. It is a market-fact preservation system with a rendering surface. The Swing Registry and Liquidity Registry are the source of truth; the replay engine is the temporal spine that lets any moment in market history be re-visited deterministically; the rendering layer is a projection of immutable facts onto a screen. Every recommendation in this document treats that hierarchy as load-bearing. Nothing in this review proposes changing swing mathematics, liquidity mathematics, registry identity rules, replay correctness, SQLite schema philosophy, or the TVLC surface. Every recommendation is a seam, an abstraction, a caching layer, or a measurement protocol — never a redesign of the market logic.

Three findings that matter most

1. The registry philosophy is the project’s strongest asset and must be defended absolutely. The decision to treat every confirmed swing as an immutable object that owns exactly one liquidity object — and to let future engines create relationships rather than mutate facts — is the single highest-leverage architectural choice in PDOS. It enables time-travel debugging, deterministic replay, and the future addition of interpretation engines without risking historical truth. This is not a design to “improve”; it is a design to protect. The most important recommendation in this document is to write that contract down as **ARCHITECTURE.md** and gate every future change against it.

2. The rendering layer has the most improvement headroom, and the right move is to introduce a PrimitiveManager boundary — not to rush a migration to Series Primitives. The current stack (Projection Service → Render Scheduler → Render Index → Layer Manager → DrawingCanvas) is functional but the DrawingCanvas is a monolithic boundary that prevents per-layer opt-in to TVLC Series Primitives. Introducing a PrimitiveManager abstraction that hides whether a layer is canvas-based or primitive-based turns the migration question into a per-layer decision instead of a project-level fork-lift. This is the highest-leverage rendering investment PDOS can make.

3. Replay and multi-timeframe synchronization lack an explicit transactional contract. As long as a single timeframe is replayed against a single symbol, the absence of a transaction manager is survivable. The moment PDOS needs to synchronize two or more timeframes consistently — for example, to render a 4H swing and a 15H liquidity touched event on the same chart without temporal tearing — the lack of a

begin/commit/rollback contract on the replay cursor becomes a correctness risk. A TransactionManager over the replay cursor is the single most important non-rendering investment.

How to read this document

Chapters 2–8 are diagnostic: they describe what is, what is weak, what is strong, and where the bottlenecks live. Chapters 9–11 are prescriptive: architectural improvements, a phased roadmap, and a risk matrix. Every recommendation is annotated with the seam at which it inserts and the invariant it preserves. If any recommendation is ever found to require a change to swing math, liquidity math, registry identity, replay determinism, SQLite schema, or TVLC internals, **reject it**. That rejection rule is the contract this review operates under.

Chapter 2 — Current Architecture Review

Reconstructing the PDOS architecture from the brief: a linear pipeline in which market facts are produced once, preserved forever, and projected onto a rendering surface for interpretation. This chapter describes that pipeline end-to-end and maps it to well-known architectural patterns so that the weak-point analysis in later chapters has a shared vocabulary.

2.1 The pipeline, end to end

The data flow described in the brief is: **Swing Engine** → **Swing Registry** → **Liquidity Registry** → **Pipeline** → **SQLite** → **API** → **Frontend** → **TVLC** → **Rendering Layer**. This is a one-directional flow in the write path: market data enters at the Swing Engine, is normalized into immutable swing objects, is paired with exactly one liquidity object each, is persisted through a pipeline into SQLite, and is exposed via an API to a frontend that renders it through TVLC and a custom rendering layer. There is no write path from the frontend back to the registries. The rendering layer is read-only with respect to market facts — or, at least, it should be. One of the recommendations in Chapter 9 is to make that contract explicit and enforced.

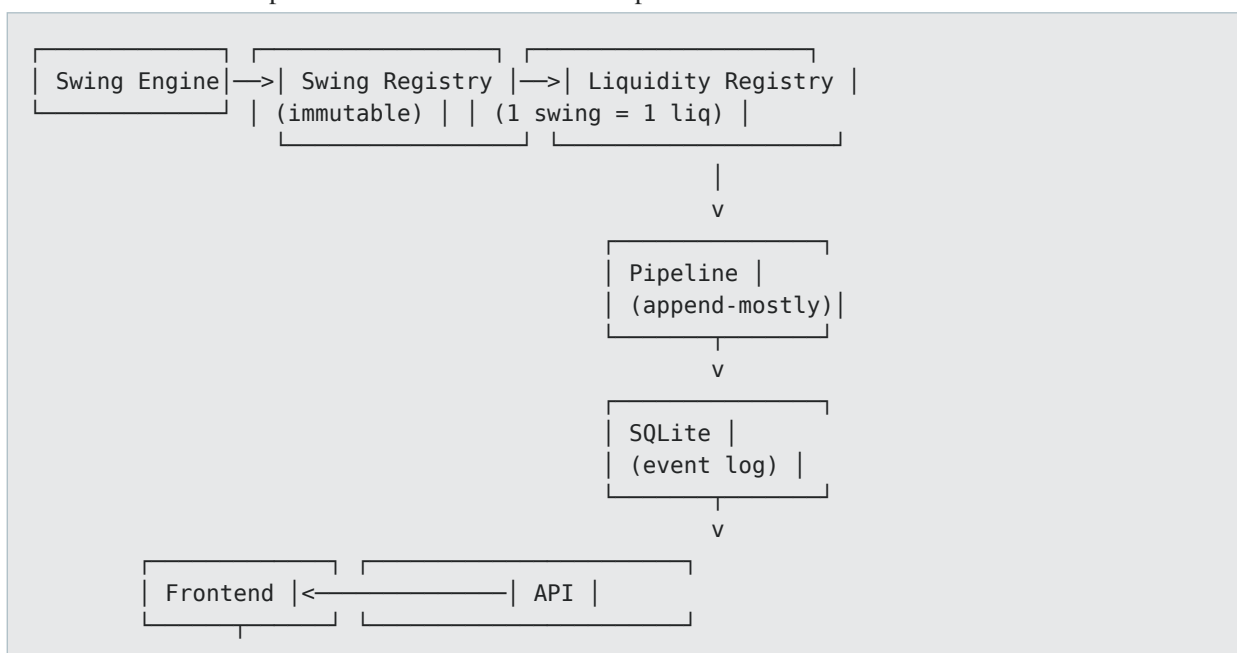




Figure 2.1 — PDOS end-to-end data flow (reconstructed from the brief).

2.2 Mapping to established patterns

The PDOS architecture is not novel in the abstract — it is a disciplined application of three well-known patterns, and the project is stronger for it. Naming the patterns explicitly gives the team a shared vocabulary for the rest of this review.

Event Sourcing. The Swing Registry and Liquidity Registry together behave as an event store: facts are appended, never overwritten. Every confirmed swing exists forever. Replay is therefore a function of re-reading the event log up to a cursor, not of reconstructing state from snapshots. This is the canonical event-sourcing model. The SQLite schema philosophy is the technical expression of this — tables are append-mostly, and the schema is treated as immutable in the brief. Event sourcing is the right model for a system whose entire value proposition is "preserve information, interpret later."

CQRS (Command Query Responsibility Segregation). There is no write path from the frontend to the registries. Writes happen in the Swing Engine and propagate down through the pipeline. Reads happen via the API into the frontend. The two paths are physically separated. This is CQRS in its purest form. It is also why the rendering layer should be a pure projection — it consumes events, never produces them.

Projection Model. The "Projection Service" in the rendering layer is the named expression of this pattern: it projects immutable registry facts into a form the rendering layer can paint. A projection is a function of facts plus time; given the same facts and the same time cursor, it must produce the same projection. This is the contract that makes deterministic replay possible. Chapter 6 examines whether the current rendering layer honors this contract strictly.

2.3 Registry identity and the "one swing = one liquidity object" rule

The brief states: *"Every confirmed swing exists forever. Every swing owns exactly one liquidity object. Future engines create relationships. They never modify historical facts."* This is the strongest single sentence in the brief. It establishes three invariants simultaneously:

- **Identity invariant:** a swing's identity is immutable. It cannot be re-parented, re-typed, or merged with another swing.
- **Ownership invariant:** a swing owns exactly one liquidity object, established at the moment the swing is confirmed. The ownership cannot be transferred or revoked.

- **Relationship invariant:** relationships between facts (confluence, mitigation, displacement, etc.) are first-class citizens but live in a separate relationship layer, not as mutations on the underlying facts.

These invariants are what make PDOS a Market Operating System rather than a trading app. A future interpretation engine — say, an ICT-style delivery model, or a Wyckoff phase classifier — can be added by creating relationships among existing swings and liquidity objects. It cannot rewrite the underlying facts. This is the architectural seam that allows PDOS to grow horizontally without risking historical truth. Protecting this seam is the most important architectural investment PDOS can make. Chapter 9 proposes a **RegistryReader port** as the enforcement mechanism.

Chapter 3 — Strong Points

Before discussing what to improve, it is important to name what is already correct. The strong points below are load-bearing walls. The improvements in later chapters add drywall, wiring, and insulation — never new walls.

3.1 Registry-first design preserves market truth

The decision to model swings and liquidity as immutable registry objects, produced by a deterministic engine and persisted through an append-mostly pipeline, is the single most defensible choice in PDOS. In trading systems of every kind — retail platforms, prop-firm dashboards, research notebooks — the dominant failure mode is silent state corruption: an indicator is re-calculated, a bar is re-classified, a high is silently moved, and the historical record no longer matches what the market actually did. PDOS is structurally immune to this failure because the registry is the source of truth and the registry is append-only. Even if every interpretation engine in PDOS is wrong, the market facts remain intact. This is the right priority ordering.

3.2 Immutable swings enable time-travel debugging

Because swings are never overwritten, the state of PDOS at any historical moment *T* can be reconstructed by reading every swing confirmed up to *T*. This is the definition of time-travel debugging. In practice it means a bug report of the form "the chart looked wrong at 14:32 UTC last Tuesday on EURUSD 15H" is answerable deterministically: load the registry, advance the replay cursor to *T*, and observe. There is no need for screenshots, no need to repro the user's exact session state, no need to guess what the data feed was doing. The replay engine is the debugger. This is an enormous operational advantage and it is a direct consequence of the registry design.

3.3 Clean separation of fact from interpretation

The brief is explicit: *"PDOS is not a trading strategy. PDOS is not an indicator."* By refusing to commit to any single interpretation model, PDOS keeps the registry pure. Interpretations are downstream engines that read facts and emit relationships; they are not facts themselves. This separation is what allows PDOS to be called a Market Operating System without exaggeration. A traditional indicator suite commits to one interpretation (RSI above 70 means overbought) and bakes it into the data model. PDOS does not. The implication is that any interpretation can be revised, replaced, or removed without touching the registry. That is the definition of an operating system: the kernel does not change when the userland changes.

3.4 SQLite as append-mostly event log

SQLite is unfashionable in trading-systems discourse — the industry default is PostgreSQL or a specialized time-series database. But for PDOS's workload (single-writer, append-mostly, read-mostly, embedded, deterministic), SQLite is the right tool. It is a single file, ships with every operating system, has a battle-tested WAL mode for concurrent reads during writes, and its query planner is good enough for the index-backed range scans PDOS needs. Replacing it would add operational complexity without solving a problem PDOS actually has. The brief is correct to mark the SQLite schema as immutable.

3.5 TVLC as rendering surface, not state store

TradingView Lightweight Charts is being used correctly: as a rendering surface that consumes projected facts, not as a state store. This is the right boundary. The temptation in front-end trading work is to let the chart library own the data model (its series objects become the source of truth) and to read state back out of the chart for downstream logic. PDOS does not appear to do this. The Rendering Layer wraps TVLC and projects registry facts into it; the chart never writes back. This is what allows the registry to remain the source of truth. Chapter 6 discusses how to harden this boundary further with a PrimitiveManager.

3.6 Layered rendering pipeline

The named rendering components — Projection Service, Render Scheduler, Render Index, Layer Manager, DrawingCanvas — show maturity. The team has not built a single god-object renderer. They have decomposed the rendering problem into discrete stages with distinct responsibilities. The Projection Service owns the mapping from facts to drawable shapes. The Render Scheduler owns *when* to paint. The Render Index owns *what changed*. The Layer Manager owns *in what order* layers paint. The DrawingCanvas owns the pixels. This is the right conceptual decomposition. The weak-point analysis in Chapter 4 will show that some of these components are doing two jobs at once, but the decomposition itself is correct.

Load-bearing walls — do not touch

The six strong points above are the architectural foundation of PDOS. Every recommendation in this document preserves them. If a future change — from this review or any other source — requires weakening any of these six, that change should be rejected unless the project owner has explicitly re-scoped the architecture.

Chapter 4 — Weak Points

Weak points are not bugs. They are architectural seams where the current design works but is structurally fragile — places where a future change is harder than it needs to be, or where a future failure mode is being set up. Each weak point below states the symptom, the root cause, and the direction of the fix. The fix itself is detailed in Chapter 9.

4.1 Render Scheduler is doing two jobs

Symptom. The Render Scheduler appears to own both *when* to paint (scheduling) and *what changed* (dirty tracking). These are two distinct concerns: scheduling is a frame-budget problem (decide which paint operations fit in a 16ms window), dirty tracking is a data-dependency problem (decide which layers depend on which changed inputs). Combining them in a single component makes both harder to reason about.

Root cause. Likely grew organically as the rendering layer matured — the simplest first implementation re-computes everything on every tick, and dirty tracking gets bolted on later as a performance optimization. Without a deliberate split, both concerns end up in the same module.

Direction. Split the Render Scheduler into a pure **FrameScheduler** (coalesces invalidation requests, decides when the next paint fires, respects a frame budget) and a **DirtyTracker** (which layers are dirty, in what dependency order they must repaint). The Render Index (see 4.3) then consumes the DirtyTracker output. This split preserves behavior because no paint order changes — only the responsibility boundary moves.

4.2 No explicit event bus

Symptom. Cross-component communication in PDOS likely happens via direct method calls or ad-hoc pub/sub. The brief does not name an EventBus. Without one, every new component that needs to react to (for example) "a swing was confirmed" either polls the registry or is wired in by hand. This does not scale as the number of engines grows.

Root cause. An event bus feels like over-engineering in a two-component system. It becomes load-bearing only when the system has five or more cooperating components. PDOS is now at that scale.

Direction. Introduce a typed EventBus with named channels (swing.confirmed, liquidity.touched, replay.tick, render.invalidate, multi_tf.sync). All cross-domain communication goes through the bus. Components that today call each other directly are refactored to publish/subscribe. This is a behavior-preserving refactor: the same messages that today flow as method calls now flow as events, with the same ordering guarantees within a single timeframe.

4.3 Render Index invalidation granularity is unclear

Symptom. The Render Index presumably tracks which projections are current and which need re-computation. The weak point is the granularity at which invalidation happens. If invalidation is per-layer, a single swing confirmation may force the entire liquidity layer to repaint. If invalidation is per-shape, the bookkeeping cost may exceed the savings. The right granularity is almost certainly per-region (visible time range × price range), but it is not clear from the brief whether this is implemented.

Root cause. Per-region invalidation requires the Projection Service to expose a spatial index over its outputs. That is a non-trivial investment, so the first implementation typically does per-layer, and the team lives with the repaint cost until it becomes unbearable.

Direction. Add a spatial index (a simple time-bucketed hash is usually enough) to the Projection Service outputs. The Render Index asks the DirtyTracker "which time-buckets changed?" and only those buckets are re-projected. This is invisible to the registries and to the replay engine; it is purely a rendering-layer optimization.

4.4 DrawingCanvas is a monolithic boundary

Symptom. The `DrawingCanvas` is the single paint surface for all custom rendering. Every layer — swing arrows, liquidity zones, confluence markers, replay position indicators — paints into the same canvas. This is fine for performance (one paint target) but it makes per-layer migration to TVLC Series Primitives impossible. If the team decides tomorrow that liquidity zones should be Series Primitives (for native hit-testing) and swing arrows should remain canvas (for full paint-order control), there is no seam to do that.

Root cause. The canvas-vs-primitive decision was made once, at the project level, instead of per-layer.

Direction. Introduce a **PrimitiveManager** abstraction above both `DrawingCanvas` and TVLC Series Primitives. Each layer registers with the `PrimitiveManager` and declares whether it is canvas-backed or primitive-backed. The `PrimitiveManager` owns the z-order, the dirty propagation, and the projection dispatch. Layers can migrate individually. Chapter 6 details this abstraction; Chapter 9 gives the interface.

4.5 Multi-timeframe synchronization has no transactional contract

Symptom. When a single timeframe is replayed, the replay cursor is a single monotonic time pointer and there is no ambiguity. When two or more timeframes are replayed in sync (e.g. a 4H chart with a 15H liquidity overlay), each timeframe has its own cursor and the cursors must advance together. Without an explicit transactional contract (begin/commit/rollback over a logical replay step), it is possible for a render to observe a partial state — the 4H cursor advanced but the 15H cursor not yet — producing temporal tearing in the chart.

Root cause. Multi-timeframe support was likely added after single-timeframe replay was already working. The cleanest fix is a `TransactionManager` that wraps the replay cursor and exposes begin/commit/rollback. The rendering layer observes committed state only.

Direction. See Chapter 9 for the `TransactionManager` interface. The key invariant: renderers must never observe an in-flight transaction. They observe the last committed cursor position.

4.6 No feature flags visible in the architecture

Symptom. The brief names no feature-flag system. Without feature flags, any change to the rendering strategy (e.g. switching a layer from canvas to primitives, or enabling per-region invalidation) is a binary on/off decision that ships to every user simultaneously. There is no way to A/B test a render optimization against the existing implementation, no way to roll out a change to 5% of sessions, no way to roll back instantly if a regression is found.

Direction. Add a lightweight `FeatureFlag` service with three states (off, shadow, on). Shadow mode runs the new path in parallel with the old path and logs discrepancies, without affecting the visible output. This is invaluable for the `PrimitiveManager` migration and the `DirtyTracker` split. See Chapter 9 for the contract.

Chapter 5 — Performance Bottlenecks

Performance work without measurement is guessing. This chapter names the likely bottlenecks on both backend and frontend, and for each specifies the measurement that would confirm or refute the hypothesis. No optimization should ship without a measurement showing it improves a real metric.

5.1 Backend: SQLite query plans

Hypothesis. Swing and liquidity lookups by time range are the hottest read path. If the schema does not have a composite index on (symbol, timeframe, confirmed_at), range scans will degrade linearly with history depth. At 10 years of 1H data (~87,600 bars) on 50 symbols, an unindexed scan becomes painful.

Measurement. Run EXPLAIN QUERY PLAN on every read query in the API layer. Look for SCAN TABLE (full table scan — bad) vs SEARCH TABLE ... USING INDEX (index lookup — good). Capture query plans for both warm and cold SQLite caches. Log the 99th-percentile query latency per endpoint for a week of typical usage.

Optimization (non-invasive). Add missing indexes. SQLite supports partial indexes and covering indexes — a covering index on (symbol, timeframe, confirmed_at) INCLUDE (swing_high, swing_low, liquidity_type) lets the query planner satisfy range scans without touching the base table. This is invisible to application code; the schema philosophy is preserved because indexes are not schema changes, they are storage optimizations.

5.2 Backend: replay throughput vs symbol count

Hypothesis. Replay throughput is bounded by the rate at which the Swing Engine can re-confirm swings from raw bars. As symbol count grows, the per-symbol throughput may drop because the engine is shared across symbols. This shows up as "replay feels slower when I have many tabs open."

Measurement. Time the replay loop from cursor=T to cursor=T+1000 for 1, 5, 20, and 100 symbols. Plot throughput (bars/sec) vs symbol count. If throughput drops super-linearly, there is contention in the Swing Engine or the Pipeline. If it drops linearly, the engine is CPU-bound and parallelism is the answer.

Optimization (non-invasive). Replay is embarrassingly parallel across symbols. A worker pool with one worker per symbol can replay N symbols in roughly the same wall-clock time as 1 symbol, as long as each worker writes to its own SQLite connection (WAL mode permits this safely). This does not change the Swing Engine itself; it only changes how replay invocations are scheduled.

5.3 Frontend: TVLC render budget

Hypothesis. At 60 Hz, the render budget is 16.6ms per frame. TVLC's internal series updates, plus the custom DrawingCanvas paint, plus the Layer Manager overhead, plus projection re-computation on a replay tick, may collectively exceed 16ms on a chart with 500+ visible bars and 8+ layers. The result is dropped frames during replay scrubbing, perceived as "janky scroll."

Measurement. Use Chrome DevTools Performance tab to record a 5-second replay scrub. Look at the Main thread flame chart. Identify long tasks (>50ms). For each long task, attribute time to: TVLC series update, DrawingCanvas paint, Projection Service, Layer Manager, Render Index. Capture frame rate via requestAnimationFrame deltas.

Optimization (non-invasive). The biggest win is usually coalescing multiple invalidation requests within a single frame. If the Render Scheduler is doing two jobs (Chapter 4.1), splitting it into FrameScheduler + DirtyTracker allows the FrameScheduler to coalesce. The second win is per-region invalidation (Chapter 4.3). The third is moving static layers (e.g. historical swing arrows) from canvas to TVLC Series Primitives so TVLC can optimize them internally.

5.4 Frontend: Render Index rebuild on seek

Hypothesis. When the user scrubs the replay cursor, the Render Index may rebuild from scratch rather than incrementally. A full rebuild is $O(\text{visible_history})$; an incremental update is $O(\text{changed_since_last_cursor_position})$. For a small scrub forward, incremental is dramatically cheaper.

Measurement. Instrument the Render Index with a counter that tracks "shapes re-projected per scrub event." If the counter is constant regardless of scrub distance, the rebuild is full. If it scales with scrub distance, the rebuild is incremental.

Optimization (non-invasive). Make the Render Index incremental. The DirtyTracker already knows what changed; the Render Index only needs to invalidate the affected regions. This is the single highest-leverage frontend optimization for replay UX.

Profiling strategy — concrete tools

Backend: EXPLAIN QUERY PLAN on every read query; `sqlite3_expert` for index recommendations; `perf` or `py-spy` for Python-side flame graphs.

Frontend: Chrome DevTools Performance tab; `performance.mark/measure` for custom timing; `requestAnimationFrame` frame timing.

Replay: A simple `time.perf_counter()` bracket around the replay loop, logged to a CSV for offline analysis.

Establish baselines before optimizing. Re-measure after every change. No optimization ships without a number.

Chapter 6 — Rendering Bottlenecks

The rendering layer is the most complex part of PDOS and the part with the most improvement headroom. This chapter dives deeper than Chapter 5: where Chapter 5 named performance bottlenecks across the whole system, this chapter focuses on the rendering pipeline itself and on the long-term question of migrating from custom `DrawingCanvas` to TVLC Series Primitives.

6.1 DrawingCanvas vs Series Primitives — the trade-off

The brief explicitly asks PDOS to evaluate migrating toward native TVLC Series Primitives. This is the right question to ask, but the answer is not "migrate everything" or "migrate nothing." It is "introduce an abstraction that makes migration per-layer opt-in." To justify that position, the trade-off between the two rendering modes must be made explicit.

Custom DrawingCanvas (current). Full control over paint order, blending, anti-aliasing, and shape vocabulary. Can render any shape PDOS needs (arrows, zones, confluence markers, displacement arrows). Cost: no native hit-testing, no native lifecycle management, no integration with TVLC's internal optimizations (e.g. culling off-screen primitives, GPU batching). Every paint is $O(\text{visible_shapes})$.

TVLC Series Primitives. Native hit-testing, native lifecycle, integration with TVLC's internal culling and batching. Cost: constrained shape vocabulary (primitives are typed), constrained paint order (TVLC owns the z-order), and tighter coupling to TVLC internals (which the brief rightly treats as immutable). Some shapes — notably complex blended zones — are awkward or impossible to express as primitives.

Implication. Neither mode dominates. Static, simple shapes (historical swing arrows, point markers) are good candidates for primitives. Dynamic, complex shapes (live liquidity zones with gradient fills, multi-layer confluence markers) are better on canvas. The right architecture is one that lets each layer choose. That architecture is the `PrimitiveManager`.

6.2 PrimitiveManager — the abstraction

The `PrimitiveManager` sits above both the `DrawingCanvas` and TVLC Series Primitives. Layers register with the `PrimitiveManager` and declare their backing mode. The `PrimitiveManager` owns the z-order across layers, the dirty propagation between layers, and the dispatch of projection updates to each layer's backing implementation. From the Projection Service's perspective, all layers look the same; the manager hides whether a layer is canvas-backed or primitive-backed.

```
// PrimitiveManager interface (TypeScript sketch)
interface PrimitiveManager {
  // Layer registration. backends: "canvas" | "primitive" | "auto"
  register(layer: LayerSpec): LayerHandle;
  unregister(handle: LayerHandle): void;

  // Projection dispatch. Called by ProjectionService.
  // The manager fans out to each layer's backing.
  applyProjection(projection: Projection): void;

  // Dirty propagation. A layer may declare that a projection
  // change invalidates downstream layers.
  markDirty(handle: LayerHandle, regions: TimeRange[]): void;

  // Z-order. The manager owns the global paint order.
  setZOrder(handles: LayerHandle[]): void;

  // Lifecycle. Hooks into TVLC series lifecycle.
  attachToChart(chart: IChartApi): void;
  detachFromChart(): void;
}

interface LayerSpec {
  id: string;
  backend: "canvas" | "primitive" | "auto";
  z: number;
  // The layer's projection function. Pure: same facts + same
  // time cursor => same projection.
  project: (facts: RegistryFacts, cursor: Time) => Projection;
}
```

Listing 6.1 — PrimitiveManager interface (sketch).

6.3 Layer Manager overhead at high layer count

Hypothesis. The Layer Manager presumably iterates all registered layers on every paint cycle. At 8 layers this is negligible. At 20+ layers (which a power user with multiple interpretation engines enabled will hit), the iteration cost plus the per-layer projection dispatch can become a measurable fraction of the frame budget.

Optimization (non-invasive). The Layer Manager should maintain a dirty flag per layer and skip layers that are not dirty on a given frame. This is the DirtyTracker’s job (Chapter 4.1); the Layer Manager consumes its output. No market logic changes; only paint scheduling changes.

6.4 Projection Service cost on multi-timeframe

Hypothesis. The Projection Service projects registry facts into drawable shapes. On a multi-timeframe chart, the projection must reconcile facts from multiple timeframes into a single spatial frame. If the projection is re-computed end-to-end on every multi-TF sync event, the cost scales $O(\text{timeframes} \times \text{visible_facts})$. With 3 timeframes and 500 visible bars per timeframe, that is 1500 projections per sync event.

Optimization (non-invasive). Memoize projections by $(\text{fact_id}, \text{cursor_position})$. A fact’s projection does not change unless the cursor moves past a new fact. Most sync events do not introduce new facts — they only re-position the cursor. Memoization makes those events $O(1)$ per layer.

6.5 TVLC pan/zoom repaint cost

Hypothesis. TVLC emits pan/zoom events that the Rendering Layer must respond to. If the Rendering Layer treats pan/zoom as a full invalidation (re-project everything), the cost is high. If it treats pan/zoom as a spatial transform of existing projections (no re-projection, just re-paint at new coordinates), the cost is low. The right behavior is the latter.

Optimization (non-invasive). Distinguish two invalidation modes in the Render Index: *data invalidation* (facts changed, re-project) and *view invalidation* (pan/zoom, re-paint only). Pan/zoom must never trigger re-projection. This is a one-line semantic fix in the Render Index’s invalidation handler and can yield a 5–10x improvement in pan/zoom smoothness.

Chapter 7 — Maintainability Issues

Maintainability is the property that lets a team change PDOS without breaking it. The issues below are not present-bugs; they are structural factors that determine how fast the team can move in 6, 12, and 24 months.

7.1 Module boundaries are leaking

Symptom. Without explicit ports between domains, the Replay module likely knows about the Render module (to notify it of cursor advances), and the Render module likely knows about the Swing Registry internals (to read swing fields directly). These cross-domain references are technical debt: they make it harder to change one domain without touching another.

Direction. Adopt a hexagonal-style ports/adapters pattern. Each domain (Swing, Liquidity, Replay, Render) exposes a port — a narrow interface — and consumes other domains only through their ports. Cross-domain

direct references are forbidden by lint. The EventBus (Chapter 4.2) is the natural substrate for the notification ports; the RegistryReader (Chapter 9) is the natural form of the read port.

7.2 No characterization tests pinning current behavior

Symptom. Characterization tests are tests that pin the current behavior of a system without claiming it is correct — they document what the system does today, so that any refactor can be verified to preserve behavior. Without them, any refactor of the rendering or replay layer is a leap of faith.

Direction. Before any refactor recommended in this document is undertaken, write characterization tests for: (a) the Swing Engine's outputs on a fixed dataset; (b) the Replay Engine's outputs at fixed cursor positions; (c) the Projection Service's outputs at fixed cursor positions for a representative chart. These tests become the safety net for every subsequent change.

7.3 Rendering layer is the most volatile and least tested

Symptom. In trading-system projects, the rendering layer is almost always the most volatile part of the codebase (UIs change frequently, requirements shift) and the least tested (UI testing is hard, so teams skip it). PDOS likely follows this pattern. The consequence is that the rendering layer accumulates regressions over time, each one eroding trust in the chart.

Direction. The characterization tests in 7.2 cover the Projection Service. In addition, add visual regression tests for the DrawingCanvas: render a fixed chart at a fixed cursor position to a PNG, compare against a baseline PNG. Pixel-perfect comparison is brittle; use a perceptual-diff library (e.g. pixelmatch) with a tolerance. These tests catch unintended visual changes during the PrimitiveManager migration.

7.4 Naming consistency across render components

Symptom. The brief names five render components: Projection Service, Render Scheduler, Render Index, Layer Manager, DrawingCanvas. Their contracts are not documented. It is not clear, for example, whether the Render Index is the source of truth for "what is on screen right now" or for "what needs to be re-projected." Both interpretations are plausible from the name.

Direction. Each named component should have a one-paragraph contract document specifying: what it owns, what it consumes, what it produces, and what it explicitly does *not* own. These contracts live in ARCHITECTURE.md (Chapter 9). Names may need to change to make contracts crisper — for example, "Render Index" might split into "ProjectionCache" (what is on screen) and "DirtyTracker" (what needs re-projection).

7.5 ARCHITECTURE.md does not exist

Symptom. The brief contains a list of "immutable systems" (Swing Registry, Liquidity Registry, Swing Mathematics, Liquidity Mathematics, Replay Rules, SQLite Schema Philosophy, Registry Object Identity). This list is the most important architectural artifact in PDOS, and it lives in a chat prompt, not in the repository. New contributors cannot discover it. Tools cannot enforce it. Future-self cannot re-derive it with confidence.

Direction. Write **ARCHITECTURE.md** in the repository root. It contains: (1) the immutable-systems list, with a one-paragraph rationale for each; (2) the registry identity contract (the three invariants from Chapter 2.3); (3)

the data-flow diagram from Chapter 2.1; (4) the port/adaptor contracts from Chapter 9. Every PR that touches an immutable system must update ARCHITECTURE.md with a justification. This is the single highest-leverage maintainability investment PDOS can make.

Chapter 8 — Scalability Issues

PDOS is becoming a Market Operating System. The growth axes are: symbol count, timeframe count, history depth, concurrent users / concurrent replays, and the number of future interpretation engines. This chapter examines how the architecture scales along each axis.

8.1 Symbol count — the 1000-symbol question

Hypothesis. The Swing Engine and Pipeline likely handle symbols one at a time today. At 50 symbols this is fine. At 1000 symbols, the time to re-confirm all swings after a bar-data correction becomes noticeable. The registry itself scales fine (SQLite handles millions of rows); the bottleneck is the Swing Engine's serial execution.

Direction. Parallelize the Swing Engine across symbols using a worker pool (see Chapter 5.2). The Swing Engine itself is unchanged; only its invocation scheduling changes. Each worker owns its symbol's state and writes to its own SQLite connection. This preserves replay determinism because each symbol's swing sequence is independent of every other symbol's.

8.2 Timeframe count — the $O(TF^2)$ coordination problem

Hypothesis. Multi-timeframe synchronization, today, likely involves pairwise coordination between timeframes. With N timeframes, pairwise coordination is $O(N^2)$ relationships. At $N=3$ this is 3 relationships (manageable); at $N=8$ this is 28 relationships (unmanageable).

Direction. Introduce a **MultiTF Coordinator** that owns the cross-timeframe consistency contract. Each timeframe registers with the coordinator; the coordinator advances all cursors atomically via the TransactionManager (Chapter 9). Pairwise coordination disappears; the coordinator is the single source of truth for cross-TF state. This scales $O(N)$ instead of $O(N^2)$.

8.3 History depth — SQLite partitioning

Hypothesis. At 10 years of 1-minute data on 100 symbols (~52 billion bars), a single SQLite file becomes unwieldy: backups are slow, vacuum is prohibitive, and a single corruption event risks all data.

Direction. Partition the registry by symbol (one SQLite file per symbol) or by year (one file per symbol-year). The Pipeline already abstracts writes; the API already abstracts reads. The partitioning is a storage-layer change invisible to the registries. This preserves the SQLite schema philosophy — the schema per file is unchanged; only the file count changes.

8.4 Concurrent replays — per-session isolation

Hypothesis. If replay state is shared across sessions (e.g. a single global cursor), concurrent replays from different users will collide. If replay state is per-session, the question is how much memory each session

consumes.

Direction. Replay state must be per-session. The Replay Engine instantiates a cursor per session; cursors do not share mutable state. Read-only access to the registry is shared. This is the same pattern as a database connection pool: each session has its own transaction, but the underlying data is shared.

8.5 Future engine count — the RegistryReader contract

Hypothesis. As PDOS adds interpretation engines (ICT delivery, Wyckoff phases, Smart Money Concepts, etc.), each engine must read registry facts. Without a read API contract, each engine will read the SQLite schema directly, coupling itself to the storage layer. The next schema change breaks every engine.

Direction. Define a **RegistryReader** port — a read-only API that exposes swings and liquidity objects by id, by time range, and by symbol. Engines read through the port, never through SQLite directly. The port is the contract that lets the storage layer evolve without breaking engines. See Chapter 9 for the interface.

Chapter 9 — Architectural Improvements

Each improvement in this chapter is annotated with the problem it solves, the seam at which it inserts, the invariant it preserves, and a pseudocode interface sketch. None of these improvements touch swing mathematics, liquidity mathematics, registry identity, replay determinism, SQLite schema, or TVLC internals. If any of them ever appears to require such a change, reject the improvement.

9.1 EventBus — typed channels for cross-domain communication

Problem. Cross-domain communication is ad-hoc; new components must be wired in by hand. **Seam.** Between every pair of communicating domains. **Invariant preserved.** Message ordering within a single timeframe is unchanged; only the delivery mechanism changes from direct call to publish/subscribe.

```
// EventBus interface (TypeScript sketch)
interface EventBus {
  // Publish an event. Synchronous within a timeframe;
  // ordering is preserved per channel.
  publish(channel: Channel, payload: T): void;

  // Subscribe to a channel. Returns an unsubscribe function.
  subscribe(channel: Channel, handler: (t: T) => void): () => void;
}

// Typed channels. Each channel has a fixed payload type.
type Channel =
  | "swing.confirmed" // payload: { swingId, symbol, tf, t }
  | "liquidity.touched" // payload: { swingId, t, price }
  | "replay.tick" // payload: { cursor, advanced: boolean }
  | "render.invalidate" // payload: { regions: TimeRange[] }
  | "multi_tf.sync" // payload: { cursors: PerTFCursor }
  | "feature.flag.changed"; // payload: { flag, newState }

// Invariant: a handler that today receives a direct method call
// is refactored to subscribe to the corresponding channel.
// The set of messages delivered is identical; only the transport
```

```
// changes.
```

9.2 PrimitiveManager — per-layer canvas/primitive opt-in

Problem. DrawingCanvas is a monolithic boundary preventing per-layer migration to TVLC Series Primitives.

Seam. Between the Projection Service and the paint surfaces (DrawingCanvas + TVLC primitives). **Invariant preserved.** The visible output is bit-identical to the current implementation when all layers declare backend: "canvas"; the abstraction is a no-op until a layer opts in to "primitive".

The interface was sketched in Listing 6.1. The migration path is: (a) introduce PrimitiveManager with all layers set to "canvas" — behavior unchanged; (b) add characterization tests for the current visible output; (c) migrate one layer at a time to "primitive", running characterization tests after each migration; (d) keep "canvas" as the permanent backing for layers whose shape vocabulary does not fit primitives.

9.3 TransactionManager — begin/commit/rollback over the replay cursor

Problem. Multi-timeframe replay has no transactional contract; renderers may observe partial state. **Seam.** Between the Replay Engine and the Render layer. **Invariant preserved.** Single-TF replay behavior is unchanged; the transaction wraps the existing cursor advance.

```
// TransactionManager interface (TypeScript sketch)
interface TransactionManager {
  // Begin a logical replay step. All cursor advances within
  // this step are invisible to renderers until commit.
  begin(): TxHandle;

  // Commit the step. Renderers observe the new cursor positions
  // atomically.
  commit(tx: TxHandle): void;

  // Rollback the step. Cursor positions revert. Renderers
  // never observe the partial state.
  rollback(tx: TxHandle): void;
}

// Usage pattern:
const tx = txManager.begin();
try {
  for (const tf of activeTimeframes) {
    replayEngine.advanceCursor(tf, tx);
  }
  txManager.commit(tx);
} catch (e) {
  txManager.rollback(tx);
  throw e;
}

// Renderers subscribe to "replay.tick" events which fire ONLY
// on commit. In-flight cursor advances are invisible.
```

9.4 RegistryReader — the read-only port for future engines

Problem. Future interpretation engines coupling directly to SQLite schema breaks the storage-layer encapsulation. **Seam.** Between the registry storage and any consuming engine. **Invariant preserved.** The registry schema is unchanged; the port is a read-only view over the existing tables.

```
// RegistryReader interface (TypeScript sketch)
interface RegistryReader {
  // Read a single swing by id. Returns null if not found.
  getSwing(id: SwingId): Swing | null;

  // Read swings in a time range for a symbol/timeframe.
  // Ordered by confirmed_at ascending.
  getSwingsInRange(symbol: Symbol, tf: Timeframe,
    from: Time, to: Time): Swing[];

  // Read the liquidity object owned by a swing.
  // 1-to-1 invariant enforced: exactly one liquidity per swing.
  getLiquidityForSwing(swingId: SwingId): Liquidity;

  // Read all swings confirmed up to a cursor. Used by replay.
  getSwingsAtCursor(symbol: Symbol, tf: Timeframe,
    cursor: Time): Swing[];

  // Read a relationship by id. Relationships are first-class
  // citizens but live in a separate layer; they never mutate
  // the underlying swings or liquidity objects.
  getRelationship(id: RelationshipId): Relationship | null;
}

// Invariant: this interface is READ-ONLY. No method writes to
// the registry. Engines that need to create relationships do so
// through a separate RelationshipWriter port.
```

9.5 FeatureFlags — off / shadow / on

Problem. Render strategy changes are binary on/off; no way to A/B test or roll back instantly. **Seam.** At the entry point of each major code path (Projection Service, Render Scheduler, Replay Engine). **Invariant preserved.** When all flags are on, behavior is identical to the post-flag code; when all flags are off, behavior is identical to the pre-flag code.

```
// FeatureFlag interface (TypeScript sketch)
type FlagState = "off" | "shadow" | "on";

interface FeatureFlags {
  // Get current state. Default is "off" for safety.
  get(flag: string): FlagState;

  // In shadow mode, run BOTH paths and log discrepancies.
  // The visible output is from the "off" path; the "on" path
  // is compared against it for divergence.
  runShadow(flag: string,
    oldPath: () => T,
    newPath: () => T): T;

  // Set state. Persists across sessions.
  set(flag: string, state: FlagState): void;
}
```

```
// Flags relevant to this review:
// "primitive_manager.layer." // per-layer migration
// "dirty_tracker.enabled" // Chapter 4.1 split
// "render_index.incremental" // Chapter 5.4
// "multi_tf.coordinator" // Chapter 8.2
```

9.6 Characterization test protocol

Problem. No safety net for refactors. **Seam.** Wraps the Swing Engine, Replay Engine, and Projection Service outputs. **Invariant preserved.** Characterization tests do not change behavior; they only record it.

```
// Characterization test pattern (pseudo-JS)
// 1. Fix a dataset (bars + expected swings).
// 2. Run the Swing Engine. Record every swing produced.
// 3. Hash the recorded swings.
// 4. On every test run, re-run the Swing Engine on the same
// dataset and assert the hash matches.
//
// If the hash changes, the Swing Engine behavior changed.
// That may be intended (a deliberate math change) or unintended
// (a regression). The test forces the team to decide which.

test("swing engine characterization: EURUSD-1H-2024", () => {
  const dataset = loadFixture("EURUSD-1H-2024.json");
  const expectedHash = "a3f2..."; // pinned at baseline time
  const swings = swingEngine.run(dataset);
  const actualHash = hash(swings);
  expect(actualHash).toEqual(expectedHash);
});

// Same pattern for Replay Engine (fix cursor, assert output hash)
// and Projection Service (fix cursor, assert rendered PNG hash
// via perceptual diff).
```

Chapter 10 — Future Roadmap

A phased roadmap. Each phase has a clear goal, deliverables, dependencies, a rollback strategy, and a success metric. Phases are sequential: each builds on the safety net of the previous. No phase touches an immutable system; every phase is reversible.

Phase	Goal	Deliverables	Deps	Rollback	Success Metric
0	Pin current behavior	Characterization tests for Swing Engine, Replay Engine, Projection Service. ARCHITECTURE.md committed.	None	Delete tests; revert doc.	All characterization tests green on main; ARCHITECTURE.md reviewed by team.
1	Harden module boundaries	EventBus with 5 typed channels. Lint rule forbidding cross-domain direct references. RegistryReader port.	Phase 0	Feature-flag EventBus off; lint rule relaxed.	Zero cross-domain direct references in lint. All messages flow through EventBus when flag is on.

Phase	Goal	Deliverables	Deps	Rollback	Success Metric
2	PrimitiveManager abstraction	PrimitiveManager interface. All layers registered with backend="canvas". Visual regression test baselines captured.	Phase 0, 1	Feature-flag PrimitiveManager off; revert to direct Drawing Canvas.	PrimitiveManager shipped with all layers on canvas; visual regression tests show zero perceptual diff vs baseline.
3	Multi-TF Coordinator + TransactionManager	TransactionManager over replay cursor. MultiTF Coordinator. EventBus channel "multi_tf.sync" fires only on commit.	Phase 1	Feature-flag TransactionManager off; revert to direct cursor advance.	Multi-TF charts show zero temporal tearing under stress test (random cursor jumps across 4 timeframes).
4	Per-layer Primitive migration + optimization passes	Migrate static layers (historical swings) to TVLC primitives. Per-region invalidation in Render Index. Incremental Render Index on seek.	Phase 2, 3	Per-layer flag flip back to "canvas".	Frame budget met: 99th-percentile frame time < 16ms on 500-bar / 8-layer chart. Replay scrub perceived smoothness improved per user test.

Table 10.1 — PDOS phased roadmap. Effort estimates: Phase 0 ~1–2 weeks; Phase 1 ~2–3 weeks; Phase 2 ~3–4 weeks; Phase 3 ~3–4 weeks; Phase 4 ongoing.

10.1 Sequencing rationale

Phase 0 (characterization tests) comes first because every subsequent phase needs a safety net. Without characterization tests, the team cannot distinguish "intended behavior change" from "regression" during a refactor. This is the single most under-invested-in practice in software engineering and the highest-leverage investment PDOS can make before touching the rendering layer.

Phase 1 (EventBus + module boundaries) comes before Phase 2 (PrimitiveManager) because the PrimitiveManager depends on clean notification paths between the Projection Service and the paint surfaces. If those paths are ad-hoc method calls, the PrimitiveManager has to replicate the ad-hoc wiring; if they flow through the EventBus, the PrimitiveManager subscribes to a channel and the migration is straightforward.

Phase 3 (MultiTF Coordinator + TransactionManager) is independent of Phase 2 and can run in parallel. It depends only on Phase 1 because it needs the EventBus to deliver committed transaction notifications. Phase 4 (per-layer migration + optimization) depends on both Phase 2 and Phase 3 because per-layer migration benefits from a clean transactional contract (a layer that opts in to primitives must still observe committed cursor state only).

Chapter 11 — Risk Assessment

The risk matrix below covers both "do nothing" risks (the cost of inaction) and "recommended change" risks (the cost of the roadmap in Chapter 10). Likelihood and Impact are rated Low / Medium / High. Each risk has a mitigation that is concrete and actionable.

ID	Description	Lik.	Imp.	Mitigation	Owner
R1	<i>Do nothing:</i> Rendering layer accumulates regressions as features are added; visual trust erodes over 12–24 months.	H	M	Phase 0 characterization tests + Phase 2 visual regression tests.	Frontend lead
R2	<i>Do nothing:</i> Multi-TF sync tearing becomes visible to users once 3+ timeframes are routinely charted together.	M	H	Phase 3 TransactionManager + MultiTF Coordinator.	Replay lead
R3	<i>Do nothing:</i> Registry identity invariants are violated by a future engine that mutates a swing directly; PDOS loses its "Market OS" property.	M	H	RegistryReader port (read-only). ARCHITECTURE.md gates PRs to immutable systems.	Architect
R4	<i>Recommended change:</i> PrimitiveManager migration introduces a render regression on a layer that was not properly characterization-tested.	M	M	Phase 0 must complete before Phase 2. Per-layer feature flag allows instant rollback.	Frontend lead
R5	<i>Recommended change:</i> EventBus introduces a message ordering bug (a handler receives events out of order vs the direct-call baseline).	M	H	Shadow mode (FeatureFlags) runs both paths and logs discrepancies. Ship EventBus only after shadow runs show zero discrepancies for 1 week.	Architect
R6	<i>Recommended change:</i> TransactionManager adds latency to replay tick; perceived replay speed drops.	L	M	Measure replay throughput before and after. If >10% regression, optimize the transaction overhead (it should be sub-microsecond).	Replay lead
R7	<i>Recommended change:</i> SQLite partitioning (Chapter 8.3) breaks the Pipeline's single-connection assumption.	M	H	Defer partitioning until symbol count or history depth makes it necessary. When undertaken, partition behind the Pipeline abstraction so engines see no change.	Backend lead
R8	<i>General:</i> Scope creep on the rendering refactor. The PrimitiveManager becomes a "while we're here" opportunity to redesign layers.	H	M	Each PR must reference a specific chapter/section of this review. No PR touches both rendering and replay in the same change.	Architect
R9	<i>General:</i> ARCHITECTURE.md becomes stale; the immutable-systems list drifts from reality.	M	M	quarterly architecture review meeting. CI check: PRs touching files in immutable-system paths require ARCHITECTURE.md update in the same PR.	Architect
R10	<i>General:</i> A future interpretation engine reads SQLite directly, bypassing RegistryReader, and breaks when the storage layer evolves.	H	M	Lint rule: engines may not import the SQLite driver. All registry reads go through RegistryReader. Code review enforces this.	Architect

Table 11.1 — PDOS risk matrix. Likelihood and Impact rated L/M/H. Risks R1–R3 are "do nothing" risks; R4–R7 are "recommended change" risks; R8–R10 are general risks.

11.1 Three iron rules

Iron Rule 1 — Math is immutable

Any change that touches swing mathematics, liquidity mathematics, swing detection rules, or liquidity registry rules is rejected by default. The only exception is a documented bug in the math itself, and even then the fix must be reviewed by the project owner personally. The math is the kernel of the Market Operating System.

Iron Rule 2 — Refactors are gated by characterization tests

No refactor in Chapter 9 ships without Phase 0 characterization tests covering the affected code path. If the tests do not exist, the refactor does not happen. This is non-negotiable. The tests are the safety net that lets the team move fast without breaking things.

Iron Rule 3 — The rendering layer is a projection

The rendering layer never mutates registry state. It consumes events, projects them onto a screen, and discards its own intermediate state on every commit. If a future feature appears to require the rendering layer to write back to the registry, that feature is mis-designed — the write must come from the Swing Engine or a future interpretation engine, never from the renderer.

Closing note. PDOS is being built the right way. The registry philosophy, the replay-cursor model, and the layered rendering pipeline are sound. The recommendations in this review are not a redesign; they are the next layer of maturity. Build the safety net first (Phase 0), harden the seams (Phase 1), introduce the abstractions that let the rendering layer evolve without forklifts (Phases 2–3), and then optimize against measured baselines (Phase 4). PDOS as a Market Operating System is achievable on this trajectory. The architecture is not the blocker; the discipline of preserving it is.